

BULLET STORM

演算法設計

Algorithm Design & Internal Mechanics

Project Motivation · System Architecture · Technologies Used · Experimental Results · Future Improvements

8

彈幕演算法

4

運動類型

5

技能樹

Lv 1-99

EXP 系統

專案動機



數學密集型運算挑戰

三角函數、向量、參數方程式在每幀執行數百次—— $60\text{fps} \times 400 \text{ 顆子彈} = \text{每秒 } 24,000 \text{ 次物理計算}$



碰撞偵測的效能壓力

子彈 $\times$ 玩家 $\times$ 敵人的 $O(n^2)$ 判定，需透過早期終止與延遲清除讓每幀維持在 1ms 以內



敵人行為的演算法設計

入場路徑、巡邏 AI、多階段射擊時機，全都需要有狀態的決策邏輯驅動

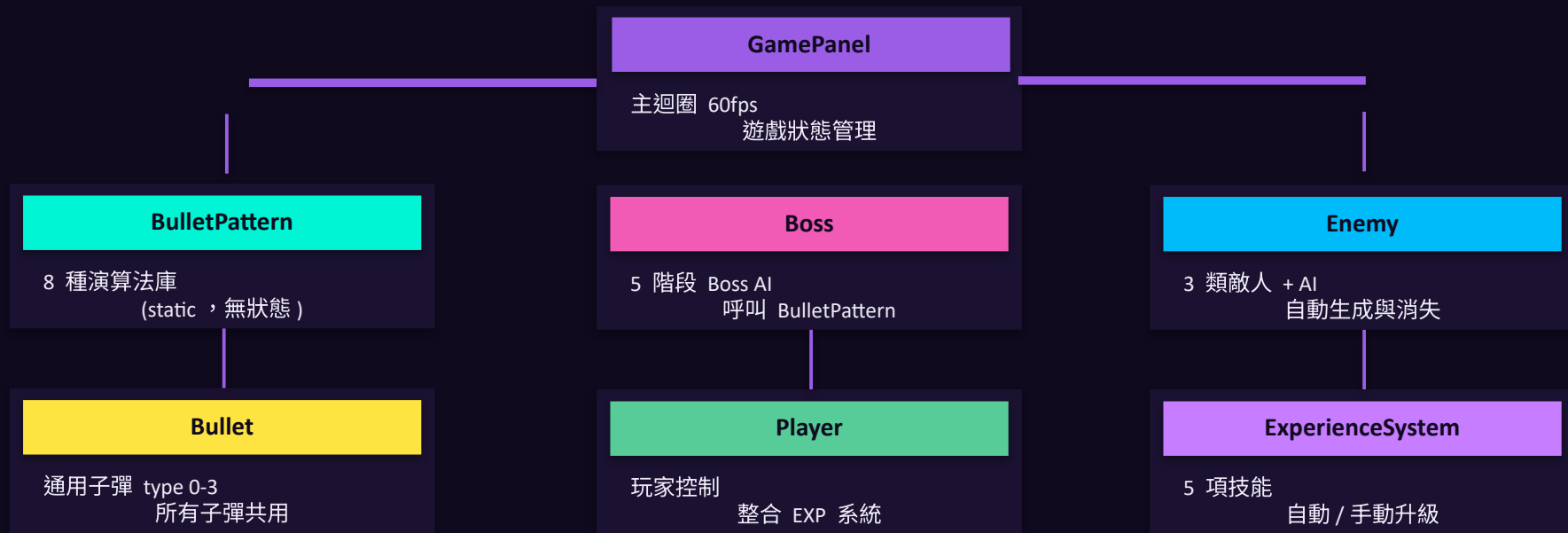


難度的多維度平衡

速度、數量、頻率三個軸同步調整，設計出難度曲線平滑遞增、不讓玩家感覺突然跳難的系統

目標：僅用標準 Java，以可組合的靜態演算法庫實現 8 種彈幕 $\times$ 4 種物理 $\times$ 5 種技能升級

核心類別架構與物件關係



BulletPattern 為純靜態工具類（無實例化），*Enemy* 與 *Boss* 傳入參數即回傳 *List<Bullet>*，達成演算法複用與解耦。

8 種彈幕生成演算法



環形爆散

`circularBurst()`

$$\theta_i = \theta_0 + i \times (2\pi/n)$$

全向 n 發



雙環交轉

`doubleRing()`

$$\text{outer_v} > 0, \text{inner_v} < 0$$

外快內慢對向



追蹤彈

`homingBullets()`

$$\text{angle} = \text{lerp}(\text{angle}, \text{target}, 0.05)$$

自動追蹤玩家



星爆交錯

`starBurst()`

$$i\%2==0 \rightarrow \text{fast}, i\%2==1 \rightarrow \text{slow}$$

交替速度環



瞄準扇射

`aimedSpread()`

$$\text{base} = \text{atan2}(\Delta y, \Delta x) \pm \text{spread}$$

鎖定玩家扇形



螺旋臂

`spiralArm()`

$$\text{offset} += \Delta\theta \text{ per frame}$$

每幀遞增旋轉



波動彈

`waveBullets()`

$$\text{offset} = \sin(\text{phase}) \times \text{amp}$$

蛇形軌跡



蝴蝶散射

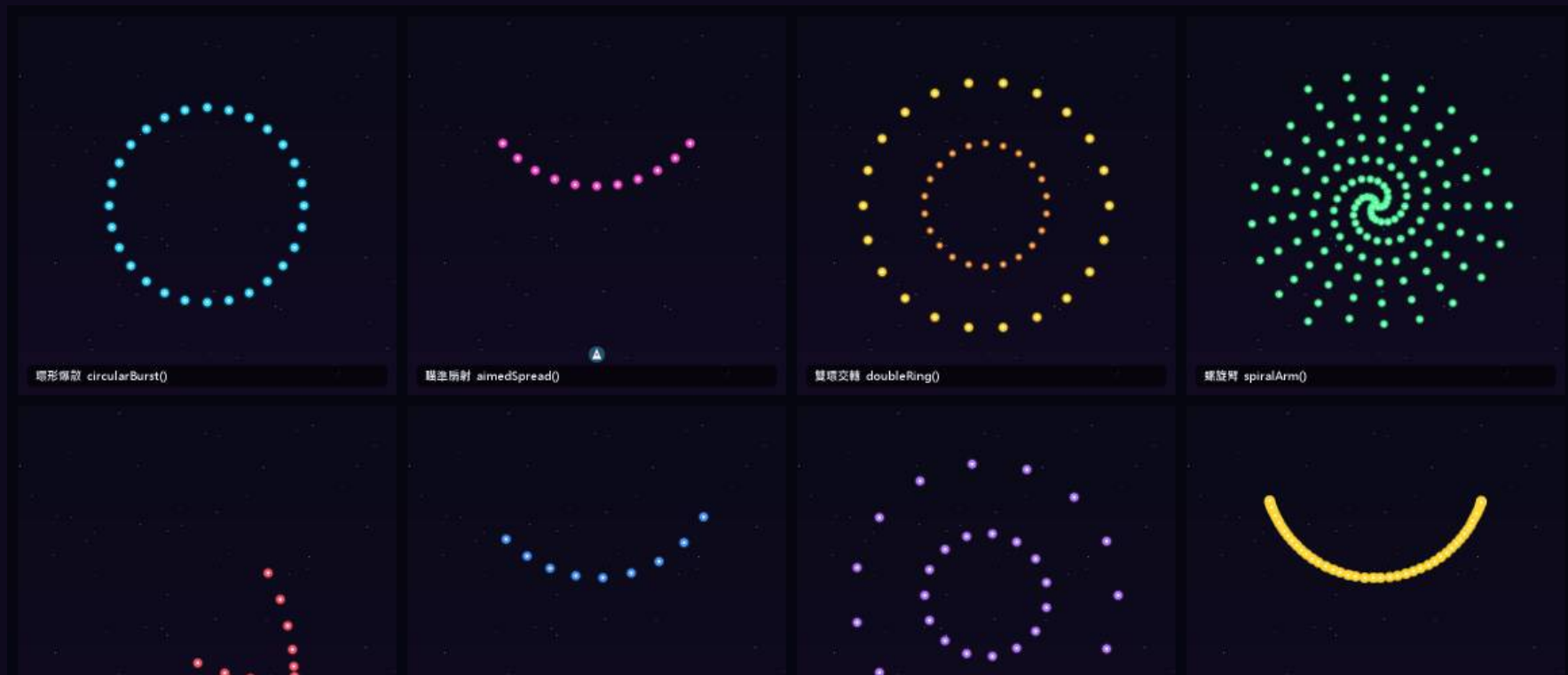
`butterflySpray()`

$$\text{spread} = \sin(t) \times \pi \times 0.4$$

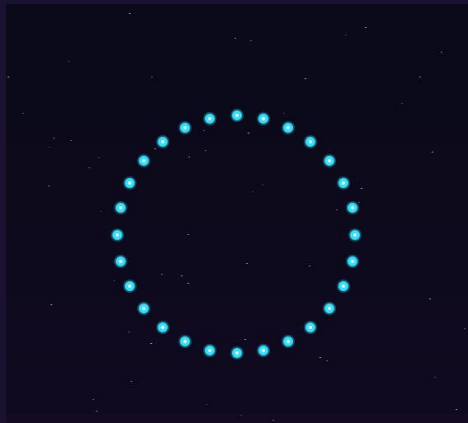
蝴蝶型 8 字

演算法實機畫面：8 種彈幕

以下皆由遊戲原始 BulletPattern 演算法即時渲染，套用遊戲相同的發光彈芯繪製。



重點演算法特寫

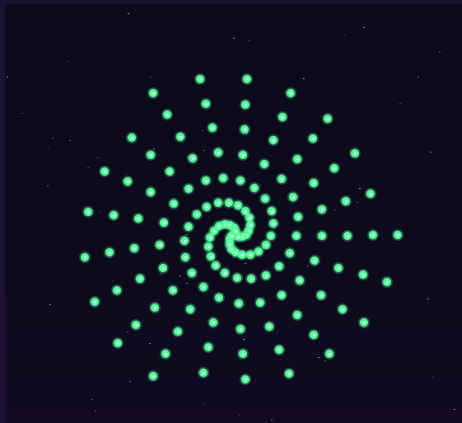


環形爆散 circularBurst()

● 環形爆散

$$\theta_i = \theta_0 + i \times (2\pi/n)$$

n 發等角灑成整圈



螺旋臂 spiralArm()

● 螺旋臂

offset += $\Delta\theta$ / frame

逐幀偏移→旋轉螺旋



追蹤彈 homingBullets()

● 追蹤彈

angle = lerp(a, target, 0.05)

每幀朝玩家修正

螺旋臂為連續多幀疊加結果；追蹤彈呈現朝右下角玩家收斂的彎曲軌跡。

子彈物理模式與 EXP 升級系統

4 種子彈運動模式（Bullet.type 欄位）

type 0

直線

```
x += vx
y += vy
```

等速直線

type 1

追蹤

```
angle → atan2(target)
clamp(Δangle, ±ω)
```

漸進轉向

type 2

波動

```
phase += freq
perp = sin(phase) * amp
```

正弦側移

type 3

加速

```
speed += accel
```

v_x, v_y 依 speed 更新
可負 accel

EXP 升級系統（5 項技能樹）

升級所需：

```
expNext = 1000 + level × 500
```

擊殺 EXP：

```
50 × (type+1) × (diff+1)
```

Boss EXP：

```
5000 × (diff+1)
```

刷掠 EXP：

```
5 / bullet
```

射速技能：

```
speed × (1.0 + lvl × 0.15)
```

射頻技能：

```
cooldown -= lvl × 2 (min 2)
```

生命技能：

```
max_life += lvl
```

炸彈技能：

```
bomb_dmg × (1 + lvl × 0.2)
```

刷掠技能：

```
graze_r × (1 + lvl × 0.1)
```

敵人 AI 生命週期管理



各敵人存活時間

敵人類型	maxAge (幀)	存活秒數	射擊間隔
type 0 仙子	480	8.0 秒	100 幀
type 1 妖怪	600	10.0 秒	90 幀
type 2 頭目	720	12.0 秒	70 幀

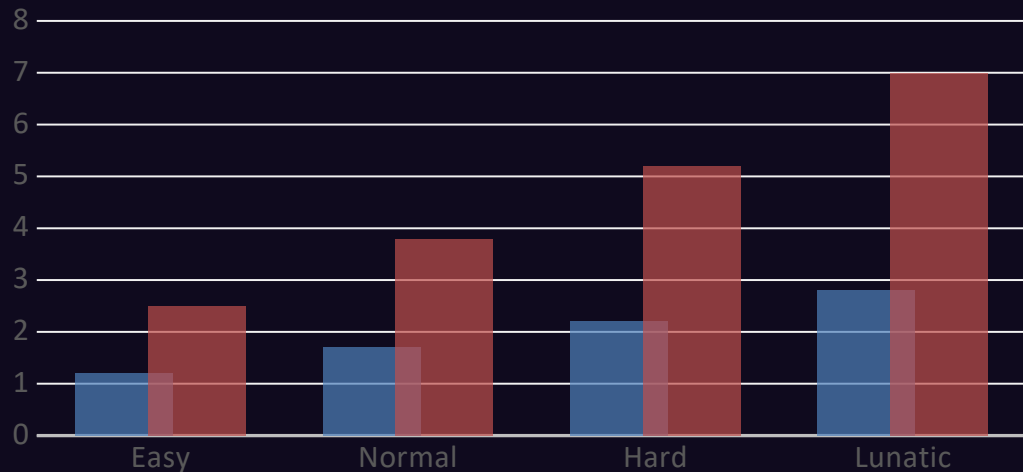
錯開生成演算法

```
for (int i = 0; i < n; i++) {
    Enemy en = new Enemy(...);
    en.age = -(i * 15);
    // 每隻差 15 幀入場
    enemies.add(en);
}

// update() 中
if (age <= 0) return;
if (age >= maxAge) {
    active = false;
}
```

實驗結果：效能與難度平衡

各難度每秒產生子彈數量



難度公式： $speedMult = (1.0 + difficulty \times 0.2) \times 0.65$ | Easy 0.65x / Normal 0.78x / Hard 0.91x / Lunatic 1.04x

最大同時子彈數

~ 400 顆

碰撞偵測每幀耗時

< 1 ms

幀率穩定度 $\pm 1\%$

60 FPS

記憶體占用

< 80 MB

未來改進方向

短期 Short-term

- **空間網格分割 (Grid)**
碰撞偵測由 $O(n^2)$ 降至 $O(n \log n)$
- **物件池 (Object Pool)**
減少 Bullet 物件 GC 壓力
- **Bezier 曲線彈道**
更自然的參數化子彈路徑

中期 Mid-term

- **難度自適應**
依存活率動態調整子彈密度
- **Boss AI 狀態機**
Idle / Attack / Enrage 三態切換
- **10+ 種技能樹**
更深度的 RPG 式成長系統

長期 Long-term

- **強化學習難度 (RL)**
以 RL 訓練最佳難度曲線
- **程序生成 Boss 技能**
隨機但平衡的攻擊組合
- **多人彈幕同步**
分散式狀態一致性演算法